

Test Technique Full Stack

Datahub Simplifié

Contexte

Vous devez développer une application web inspirée de Datahub permettant de gérer des imports de données, explorer leur contenu et produire des statistiques. La solution doit être industrialisable, maintenable et pensée pour de forts volumes de données.

Objectifs évalués

Architecture, FastAPI, Pandas, MongoDB, React TypeScript, qualité de code, Docker, performance, UX, tests et capacité d'analyse.

Technologies imposées

Backend: Python 3.13+, FastAPI, Pandas, MongoDB. Frontend: React + TypeScript. Docker et Docker Compose. Tests: Pytest, Vitest, React Testing Library.

Principes

Respect des principes KISS, SOLID, DRY, Clean Code, séparation des responsabilités et architecture modulaire et évolutive.

Fonctionnalités détaillées

Page d'accueil : liste des imports vide au démarrage. Création, édition, suppression et réorganisation des imports. Cliquer sur un import permet de l'ouvrir.

Chaque import possède deux onglets : Données et Statistiques.

Onglet 1 – Données

Import CSV/XLSX et réimport possible.

Détection automatique des types côté backend via endpoint dédié : boolean, integer, float ou string. Les valeurs vides doivent être supportées. Si le type est indéterminable alors string est utilisé.

Le tableau doit être entièrement dynamique : colonnes construites depuis les en-têtes du fichier.

Pagination rapide, optimisée et autorisée : 10, 20, 50, 100, 1000, 5000, 10000, 1 000 000 lignes.

Chaque colonne du tableau doit être triable et filtrable.

Toute action de pagination, filtrage ou tri doit déclencher un appel API. **Aucune conservation de la totalité des données côté front.**

Chaque ligne doit posséder un bouton Editer ouvrant une popup d'édition avec validation, annulation puis confirmation avant sauvegarde.

Après sauvegarde, le tableau est rafraîchi (avec un appel à l'API donc) en conservant page, tri et filtres courants. Si l'utilisateur recharge la page ou quitte l'import pour y revenir plus tard, il doit retrouver les filtres, les tris et la page avec lesquels il a quitté l'import.

Fonctionnalités batch : sélection multiple, suppression multiple et édition multiple. Pour chaque champ : conserver, modifier ou supprimer la valeur.

Onglet 2 – Statistiques

Sélection d'une colonne depuis un menu déroulant, puis calcul des statistiques côté backend.

Booléen : count global, count true, count false, répartition en pourcentage.

String : count global et affichage de tableau équivalent à celui du 1er onglet avec 2 colonnes valeur / occurrence, paginé, filtrable et triable. Tri décroissant des occurrences par défaut.

Integer ou Float : count, min, max, moyenne, plus tableau valeur / occurrence paginé.

Checkbox 1 : appliquer les filtres du tableau de l'onglet Données avant calcul des statistiques.

Checkbox 2 (lorsqu'une colonne de type string est choisie): appliquer en plus les filtres du tableau statistique valeur / occurrence sur les statistiques calculées.

Contraintes techniques

Les traitements et calculs doivent être réalisés côté backend.

Le frontend ne doit jamais charger l'intégralité du dataset.

Chaque changement de filtre, tri ou page ou de calcul statistique doit entraîner une requête backend.

Volumétrie et performance

La solution doit être pensée pour fonctionner avec jusqu'à 1 000 000 de lignes par import.

Le candidat doit documenter ses optimisations : index MongoDB, pagination serveur, agrégations MongoDB, projections, debounce frontend, virtualisation de tableaux, streaming d'import, etc.

De la même manière, le code doit être commenté de manière à expliquer pourquoi tel ou tel choix a été fait.

Architecture attendue

Le code métier ne doit pas être implémenté directement dans les routes FastAPI ni dans les composants React.

Exemple : api, services, repositories, models, schemas côté backend ; pages, composants, hooks, services, types côté frontend.

Dockerisation

Un Dockerfile et un unique docker-compose.yml doivent permettre de lancer toute la plateforme.

Le comportement doit être piloté par des fichiers .env (dev, preprod, prod).

Tests

Backend : tests unitaires pour détection de types, filtres, pagination, statistiques, batch update et tests d'intégration des endpoints FastAPI.

Frontend : tests unitaires des composants, hooks et services ainsi que tests d'intégration des parcours utilisateurs.

Couverture cible minimale : 70%.

Documentation obligatoire

README contenant architecture, choix techniques, instructions de lancement, stratégie de tests, optimisations, limitations connues.

Diagramme d'architecture obligatoire.

Justification détaillée des index MongoDB créés.

Livrables

Repository Git complet (partagé depuis gitlab.com, github.com ou équivalent ou dans un zip), code source, README, Dockerfile, docker-compose.yml et jeux de tests.

Il est demandé de voir les différents commit dans l'historique git afin de comprendre la construction et les choix pendant la construction de l'application.

Critères d'évaluation

Architecture 20%, Backend 20%, Frontend 20%, Qualité du code 15%, Tests 15%, Docker 5%, Performance 5%.